

EVERYDAY KITCHEN

Master Product + AI Architecture Plan

From kitchen inventory tracking to personalized, waste-aware cooking decisions

Document type	Living project blueprint
Current working name	Everyday Kitchen
Status	Planning + foundation setup
Version	1.0 September 2026

Name status: Everyday Kitchen is the current working product name. It has not been treated as legally cleared; a formal trademark/domain clearance should happen before commercial launch.

How to use this document: This is the single project blueprint. Update it when an architectural decision, product decision, scope change, experiment result, or major learning occurs.

Contents

1. Executive Summary
 2. Product Definition
 3. Product Vision and Principles
 4. User Journey and Core Workflow
 5. Product Scope and Release Strategy
 6. MVP Functional Specification
 7. Data Model
 8. System Architecture
 9. Recommendation Engine
 10. Decision Support
 11. Personalization and Learning
 12. Language Model / AI Layer
 13. Technology Stack
 14. Repository Structure
 15. Development Roadmap
 16. Evaluation and Success Metrics
 17. Security, Privacy and Reliability
 18. Risks and Open Questions
 19. Decision Log
 20. Future Opportunities
 21. Immediate Next Steps
- Appendix A. Glossary
- Appendix B. Example End-to-End Scenario

1. Executive Summary

Everyday Kitchen is planned as a personal kitchen inventory and cooking decision assistant. The product begins with a simple and practical problem: people often lose track of what food they own, where it is stored, how much remains, and when it expires. That creates duplicate purchases, forgotten ingredients, and avoidable food waste.

The first release therefore focuses on reliable inventory management and expiry awareness. Once that foundation is working, the product grows into a recipe recommendation system that uses the user's existing ingredients, expiry urgency and cooking preferences to surface useful options. Over time, the recommendation system learns from what the user actually views, saves, cooks, skips and rates.

The long-term differentiator is not simply recipe search. It is the combination of kitchen state, personal cooking behavior and waste-aware decision support. The intended experience is: "Given what I have, what is likely to work well for me right now, and what should I use before it goes to waste?"

The product in one sentence

Working product statement: Everyday Kitchen keeps track of what a person has, when it needs attention, and how they cook, then helps them decide what to use and what to make next.

What Everyday Kitchen is not

- It is not an AI chatbot with a kitchen-themed interface.
- It is not primarily a generic recipe catalogue.
- It is not a system that retrains a large neural network every day.
- It is not intended to silently make decisions on behalf of the user.

2. Product Definition

2.1 Problem

Kitchen inventory is a small but persistent coordination problem. Food is distributed across fridge, freezer and pantry/storage. Quantities change as people consume ingredients. Expiry dates differ. Human memory is not a dependable inventory database. The result is forgotten food, duplicated purchases and a constant need to mentally decide what to cook.

2.2 Core product job

- Know what food is currently available.
- Know what needs attention soon.
- Know which recipes are feasible with the current inventory.
- Learn which options fit the user rather than recommending generic recipes.
- Help the user make practical trade-offs between enjoyment, effort and food utilization.

2.3 Long-term user promise

Future experience: Open the app and quickly understand what needs attention, what you can make, and why a particular option may make sense tonight.

3. Product Vision and Principles

Principle	Meaning
Useful before intelligent	The inventory and expiry experience must work without AI first.
Structured data before AI	Ingredients, quantities, recipes, events and expiry information must be represented reliably.
Recommend, do not dictate	The system should surface options and reasoning while leaving the decision to the user.
Explain the why	Recommendations should have human-readable reasons tied to observable data.
Learning should be gradual	Daily learning initially means updating user signals and memory, not daily full-model retraining.
Optimize for people and food	Recommendations should balance user fit with practical ingredient utilization and waste reduction.
Build for evidence	Each more complex model should be compared against a simpler baseline.
Keep the architecture replaceable	The recommendation model, LLM provider and deployment environment should be replaceable without rewriting the entire app.

4. User Journey and Core Workflow

The core workflow below is the backbone of the product. Every later AI feature is an extension of this loop rather than a separate product.



Figure 1. Core product workflow from inventory capture to continuous learning.

4.1 Typical user journey

Stage	User / system activity
Add	The user records food with quantity, expiry and storage location.
Maintain	The user adjusts quantity, consumes items, moves them, or removes them.
Notice	Everyday Kitchen highlights items that need attention.
Discover	The user browses recipes that fit current inventory.
Decide	The system explains why certain recipes are useful right now.
Act	The user saves, cooks, skips, edits or substitutes.
Learn	Those actions become signals for future personalization.

5. Product Scope and Release Strategy

5.1 Release philosophy

The product should be built in layers. Each layer must deliver a usable capability before the next layer adds intelligence. This protects the project from becoming an AI experiment that lacks a dependable underlying product.

Release	Primary capability	AI level
MVP	Inventory + expiry + notifications + clean data foundation	No ML required
V1	Recipe database + transparent recipe ranking + explanations	Deterministic scoring
V2	Feedback capture + user preference profile	Behavior-driven personalization
V3	ML ranking model	Learned relevance prediction
V4	Decision optimization	Trade-offs, waste minimization and planning
V5	Conversational AI	Natural-language input, explanations and interaction
V6	Long-horizon intelligence	Multi-day planning, shopping and deeper optimization

5.2 What is explicitly out of scope for the first release

- Custom large language model training.
- Neural recommendation models trained on insufficient data.
- Automated grocery purchasing.
- Complex household collaboration.
- Computer vision inventory recognition as a required path.
- Perfect nutrition, health or medical personalization.

6. MVP Functional Specification

Feature	MVP behavior
Inventory entry	Add food with name, quantity, unit, purchase date, expiry date and storage location.
Inventory maintenance	Edit, delete, consume, adjust quantity and mark items as finished.
Locations	Group by fridge, freezer, pantry/storage and other.
Expiry awareness	Show expired, expiring today, expiring soon and healthy inventory.
Notifications	Local reminders for approaching expiry dates.
Ingredient normalization	Map equivalent ingredient names to a reusable canonical ingredient record where practical.
Recipe-ready data	Store inventory in a way that can later be matched to structured recipes.
Reliable edge cases	Handle zero quantity, missing expiry, quantity changes and multiple purchases of the same ingredient.

6.1 Inventory lot concept

An ingredient and a specific purchase of that ingredient should be conceptually separate. For example, 500 ml of milk bought on one date and 1 litre bought later may have different expiry dates. The recommended design therefore uses inventory lots rather than a single “milk quantity” record.

```
Ingredient
|
+-- Inventory Lot A: 500 ml, expiry Sep 20
|
+-- Inventory Lot B: 1,000 ml, expiry Sep 27
```

Why this matters: Lot-level tracking makes expiry logic and partial consumption much more reliable and gives the recommendation engine better information about which stock should be used first.

7. Data Model

The data model is the foundation of the future recommendation and learning system. Every important user action should produce structured information that can be used later without reconstructing it from free-form text.

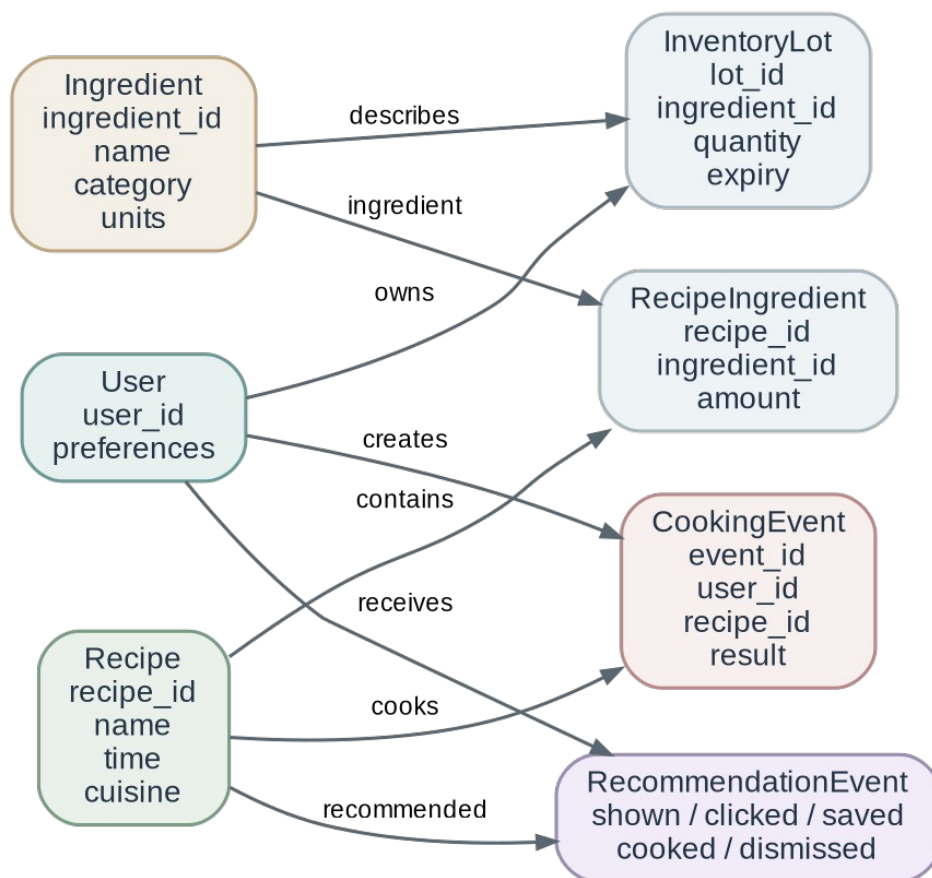


Figure 2. Initial conceptual data model and relationships.

7.1 Core entities

Entity	Purpose	Representative fields
User	Account and preference information.	user_id, household context, dietary preferences, cuisine preferences, cooking preferences
Ingredient	Canonical food item used across inventory and recipes.	ingredient_id, name, category, aliases, supported units
InventoryLot	A quantity of an ingredient acquired at a specific time with its own expiry.	lot_id, user_id, ingredient_id, quantity, unit, purchase_date, expiry_date, storage_location, status
Recipe	Structured recipe available to the recommendation system.	recipe_id, name, cuisine, time, difficulty, servings, tags
RecipeIngredient	Link between a recipe and an ingredient, including amount.	recipe_id, ingredient_id, quantity, unit, optional flag
CookingEvent	Record of a user attempting a recipe.	event_id, user_id, recipe_id, timestamp, completion, rating, modifications
RecommendationEvent	Record of what the system recommended and how the user interacted with it.	event_id, user_id, recipe_id, timestamp, shown, clicked, saved, cooked, dismissed

7.2 Future entities

- Household / shared kitchen
- Shopping list item
- Meal plan
- Ingredient substitution
- Waste event
- Recipe version / personalization state
- Notification event and delivery status

8. System Architecture

The proposed architecture separates the user interface, core application services, data, recommendation intelligence and language interaction. The AI system is deliberately downstream of reliable product data.

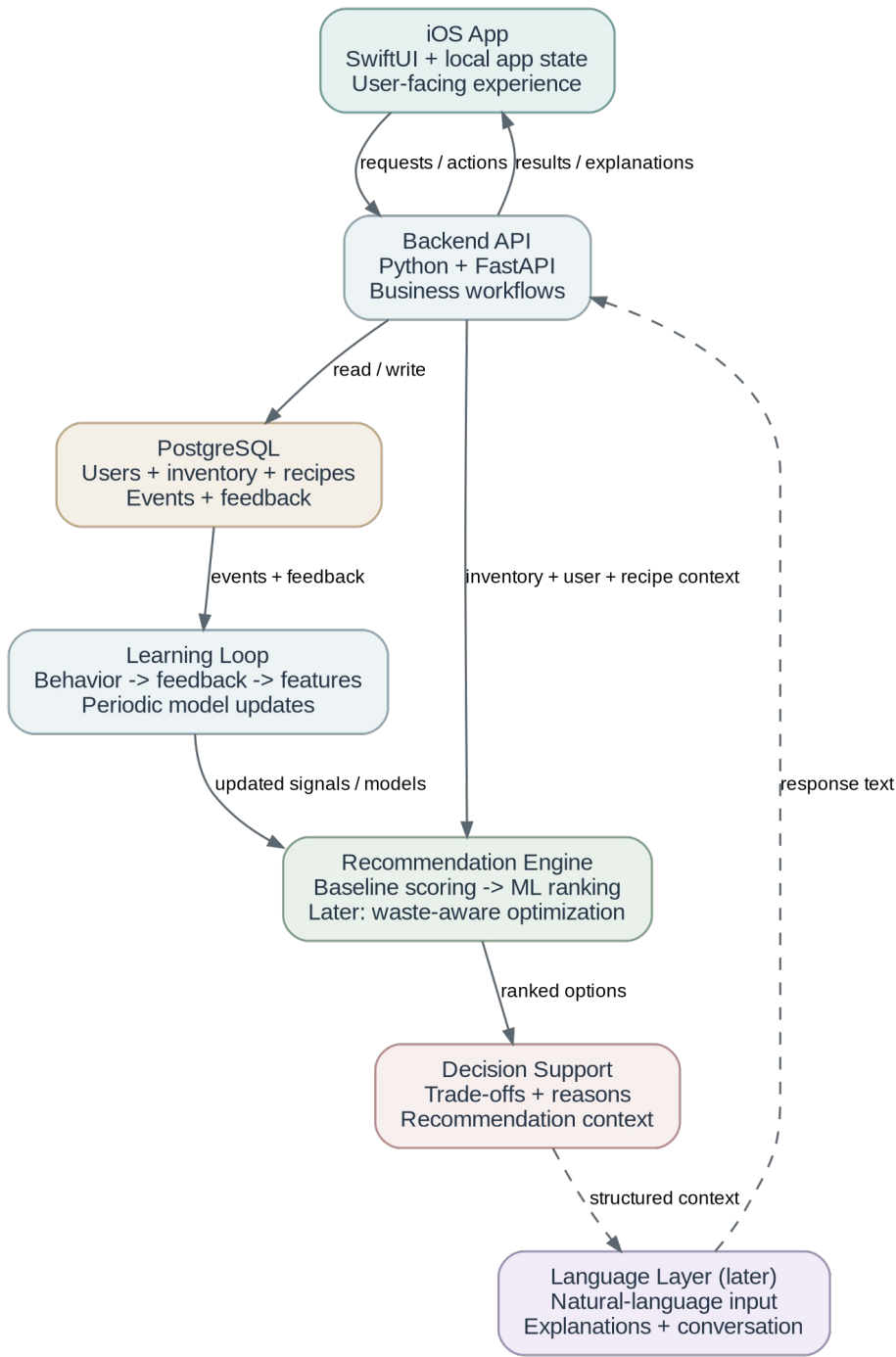


Figure 3. Planned system architecture. Components such as ML ranking, optimization and the language layer are introduced progressively.

8.1 Architecture responsibilities

Component	Primary responsibility
iOS application	SwiftUI screens, user input, inventory views, local notification scheduling, local app state/cache as appropriate.
Backend API	Authentication, inventory operations, recipe retrieval, recommendation requests, event capture and business workflows.
PostgreSQL	Persistent application data, inventory lots, recipe data, users, interactions and learning events.
Recommendation engine	Candidate selection, features, scoring, ranking and eventually learned prediction models.
Decision support	Reason generation, trade-offs, confidence/limitations and waste-aware considerations.
Language layer	Later conversational interface, structured-data extraction from natural language and natural-language explanations.
Learning system	Event pipelines, user profile updates, training datasets, model evaluation and model versioning.

8.2 Planned request flow

```

User action
-> iOS app
-> FastAPI endpoint
-> read current inventory + user profile + recipe data
-> recommendation engine
-> decision support
-> optional language layer
-> response
-> iOS app
-> user action becomes event data
  
```

9. Recommendation Engine

The recommendation engine answers: “Which recipes or actions should be surfaced to this user in this context?”
It is not required to be a neural network from the beginning.

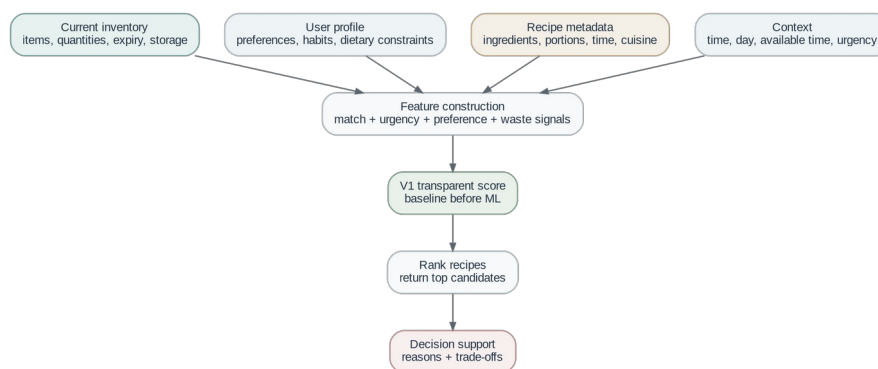


Figure 4. Recommendation pipeline from raw kitchen state to explainable ranked options.

9.1 Version 1: transparent baseline

The first recommender should be deterministic and inspectable. A weighted score can combine ingredient availability, expiry urgency, user preference, cooking-time fit, past preference and expected food waste.

```
score =
  ingredient_match      * 0.30
+ expiry_priority      * 0.25
+ preference_match     * 0.20
+ cooking_time_match   * 0.10
+ past_preference      * 0.10
+ waste_reduction      * 0.05
```

Important: The exact weights are provisional. They are starting points for experimentation, not permanent product truths. They should eventually be tuned with evidence from evaluation and real behavior.

9.2 Candidate signals

Signal	Meaning
Ingredient match	How much of the recipe can be made from what is already available?
Expiry priority	How urgently does the recipe use items that are near expiry?
Preference match	How well does the recipe align with observed or declared preferences?
Cooking effort fit	Does the recipe fit the user's typical or currently available cooking time?
Past preference	Has the user previously cooked, saved or rated similar recipes positively?
Waste reduction	How much of the existing stock would be used, and how much would likely remain?
Missing ingredient cost	How much new shopping would the recipe require?

9.3 Version 2+: machine learning ranking

Once enough interaction data exists, the same feature concepts can become inputs to a learned ranking model. The initial target can be a relevance or choice likelihood score. Model complexity should increase only when it demonstrates improvement against the baseline.

```
User features
+ Recipe features
+ Current inventory features
+ Context
  |
  v
  Ranking model
  |
  v
relevance score
```

10. Decision Support

Recommendation tells the user what to consider. Decision support makes the reasoning understandable and highlights trade-offs.

10.1 Example explanation

Example: Palak Paneer is recommended because spinach expires tomorrow, paneer expires in two days, most required ingredients are already available, the recipe fits the user's usual cooking time, and it leaves relatively little unused food.

10.2 Decision support dimensions

Dimension	Question
Urgency	Does this option use ingredients that need attention soon?
Fit	Does it match the user's preferences and constraints?
Effort	How much time and complexity does it require?
Availability	How much can be made without buying more ingredients?
Waste	How much existing stock does it use, and what is left afterward?
Confidence	How strong is the evidence behind the recommendation?

10.3 Important design rule

Decision support should expose relevant evidence without pretending to know more than the system actually knows. “You already have 5 of 6 ingredients” is stronger than an opaque claim such as “This is definitely the best choice.”

11. Personalization and Learning

The phrase “learns day to day” is interpreted as a layered learning strategy. The first layers update memory and user signals frequently. Model weight changes happen less frequently through controlled retraining and evaluation.

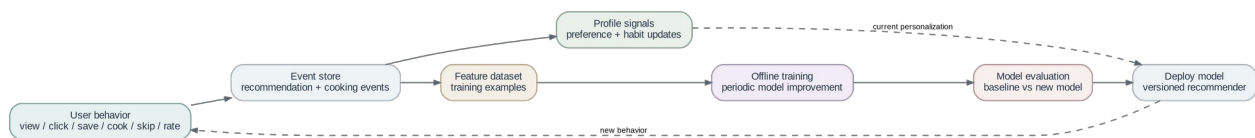


Figure 5. Learning loop: daily behavior updates personalization, while periodic training updates the model.

11.1 Daily / immediate learning

- Update preference counters and user profile signals.
- Record which recipe types are viewed, saved, cooked, skipped or rated.
- Adjust short-term context signals where appropriate.
- Use the newest behavior in future recommendation requests.

11.2 Periodic learning

- Construct training examples from historical events.

- Train a candidate ranking model offline.
- Compare the candidate model against the current baseline/model.
- Evaluate for usefulness, calibration and unwanted behavior.
- Version the model and only promote it when evaluation supports the change.

11.3 What not to do

- Do not retrain a large model every day just because new events arrived.
- Do not treat every click as a positive preference signal.
- Do not let a short-lived behavior permanently redefine the user profile.
- Do not optimize only for clicks if the product goal is practical cooking and food utilization.

12. Language Model / AI Layer

A language model should be an interaction and explanation layer around structured product logic. The language model is not the source of truth for inventory or recipe arithmetic.

12.1 Good uses for an LLM

- Convert natural-language additions into structured inventory data.
- Answer “what can I make with these?” using retrieved structured context.
- Explain why a recipe was recommended.
- Interpret flexible user requests such as “something quick and light tonight.”
- Suggest substitutions when structured ingredient knowledge supports them.
- Provide conversational access to existing product functions.

12.2 Bad uses for an LLM

- Inventing current inventory quantities.
- Calculating expiry status without using the database.
- Becoming the hidden ranking system with no evaluation.
- Changing user preferences without explicit evidence.
- Being used as a replacement for a normal database or business-rule engine.

13. Technology Stack

Layer	Current plan
iOS	Swift, SwiftUI, UserNotifications, with local persistence/cache mechanisms such as SwiftData as appropriate.
Backend	Python + FastAPI.
Database	PostgreSQL.
ML baseline	Python, NumPy, Pandas, scikit-learn.
Deep learning	PyTorch when a learned ranking model actually becomes justified.
AI / language	Embeddings/vector retrieval and an LLM later, selected according to deployment, privacy and cost constraints.
Development	Git + GitHub, Xcode, Python virtual environments, automated tests.
Deployment	Cloud provider and exact architecture to be decided after the local prototype and backend boundaries are proven.

14. Repository Structure

The repository is intentionally simple at the beginning. More specialized directories should appear when the underlying responsibilities actually exist.

```
EverydayKitchen/  
├── ai/  
├── backend/  
├── docs/  
├── ios/  
├── .gitignore  
└── README.md
```

14.1 Planned evolution

```
ai/  
├── recommendation/  
├── models/  
├── training/  
└── evaluation/  
  
backend/  
├── app/  
│   ├── api/  
│   ├── models/  
│   ├── services/  
│   └── database/  
└── tests/
```

Repository principle: Do not create dozens of empty modules now. The structure should grow with real code and responsibilities.

15. Development Roadmap



Figure 6. Planned progression from product foundation to long-horizon kitchen intelligence.

Phase	Focus	Exit condition
Phase 1	Product specification + data model	Define entities, relationships, expiry rules, recipe format and acceptance criteria.
Phase 2	Inventory + expiry	Build add/edit/consume/location flows and expiry notifications.
Phase 3	Recipes + baseline recommendation	Create a small structured recipe corpus and transparent scoring engine.
Phase 4	Feedback + personalization	Capture recommendation and cooking events and maintain user preference signals.
Phase 5	ML ranking	Train and evaluate a learned ranking model against the baseline.
Phase 6	Decision optimization	Introduce waste-aware scoring, multi-objective trade-offs and planning.
Phase 7	Language interaction	Add natural-language input, explanations and conversational workflows.
Phase 8	Long-horizon intelligence	Meal planning, shopping assistance, substitutions and deeper optimization.

16. Evaluation and Success Metrics

Success should be measured at three levels: product reliability, recommendation usefulness and food-utilization impact. Metrics should not be optimized in isolation.

Metric area	Examples
Inventory reliability	Successful add/edit/consume flows, low data-loss rate, low notification failure rate.
Recommendation interaction	Views, saves, cooking starts and completion signals.
Recommendation quality	Offline ranking metrics and human evaluation against baseline.
Personalization gain	Improvement of relevant user outcomes versus a non-personalized baseline.
Waste-aware value	Estimated amount/value of inventory used before expiry, where measurable.
User trust	Rate of meaningful interactions with explanations and low rate of “nonsense” recommendations.
System performance	API latency, failure rate, notification reliability and model inference latency.

16.1 Offline evaluation idea

For every future ranking model, keep a fixed evaluation split. Compare against the current baseline rather than only reporting the new model's score. The evaluation set should reflect realistic user behavior and time context where possible.

17. Security, Privacy and Reliability

Everyday Kitchen will eventually handle personal information about households, food habits and potentially dietary restrictions. The architecture should therefore treat privacy and data integrity as first-class concerns.

- Use secure authentication and authorization for user-owned data.
- Never commit API keys or secrets; continue using the repository .gitignore for environment files.
- Keep recommendation events tied to appropriate user permissions.
- Collect only data that serves the product or learning goals.
- Document whether data is stored locally, on the backend, or sent to external AI providers.
- Provide a plan for account deletion and data deletion before public launch.
- Treat allergies and dietary restrictions as user-provided constraints that require careful validation and safe handling.

Reliability principle: Inventory state is a source of truth. AI output is an interpretation layer. A natural-language model must never silently overwrite authoritative inventory data.

18. Risks and Open Questions

Risk / question	Why it matters
Data entry friction	If recording food is annoying, the inventory will become stale. Need fast entry workflows.
Data quality	Ingredient names, units and quantities can be messy. Need

Risk / question	Why it matters
	normalization and clear editing.
Cold-start personalization	New users have no behavior history. Baseline ranking and onboarding preferences matter.
Sparse feedback	Many recommendations may be ignored for reasons unrelated to preference. Event semantics need care.
Recipe data quality	Recommendation quality depends on structured and consistent recipe metadata.
Over-personalization	The system can become too narrow if it only repeats past behavior. Exploration will eventually be needed.
Waste objective conflict	The most waste-efficient recipe may not be the most satisfying. The product needs balanced trade-offs.
LLM reliability	Natural-language responses can hallucinate. Structured facts must come from the system of record.
Commercial name clearance	Everyday Kitchen is a working name, not a completed legal clearance.
Infrastructure cost	LLM usage, model hosting and backend scale must be evaluated after real usage is understood.

19. Decision Log

ID	Decision	Current state	Status
D-001	Working product name	Use "Everyday Kitchen" for current development and repository naming.	Current; commercial/legal clearance still open.
D-002	Repository strategy	Use one monorepo containing iOS, backend, AI and documentation.	Confirmed.
D-003	AI strategy	Build a deterministic recommendation baseline before training a neural model.	Confirmed.
D-004	Learning strategy	Daily learning means profile/event updates first; model retraining is periodic.	Confirmed.
D-005	Recommendation goal	Balance user fit with inventory utilization and future waste reduction.	Confirmed.
D-006	LLM role	Use an LLM around structured recommendation logic, not as the source of truth.	Planned.
D-007	Inventory model	Track inventory lots rather than only a single aggregate ingredient quantity.	Planned.
D-008	Immediate priority	Define data model and build the recommendation prototype before extensive UI work.	Current next step.

20. Future Opportunities

- Barcode scanning for faster inventory entry.
- Receipt scanning / OCR for batch purchase import.
- Household or shared-kitchen mode.

- Multi-day meal planning that minimizes expected waste.
- Smart shopping list generated from inventory and meal plans.
- Ingredient substitution and “missing one ingredient” suggestions.
- “Use this first” dashboard for high-risk inventory.
- Waste analytics and estimated savings over time.
- Natural-language recipe search and semantic retrieval at scale.
- Voice entry for hands-free kitchen use.
- Photo-assisted inventory recognition as an optional convenience feature.
- Offline-first behavior for core inventory actions with synchronization when connected.

21. Immediate Next Steps

The current project should remain deliberately narrow. These are the next actions, in order, before expanding scope.

1. Create docs/EVERYDAY_KITCHEN_MASTER_PLAN.md as the editable source version of this blueprint.
2. Define the database entities and relationships in detail.
3. Write exact inventory, quantity and expiry rules, including edge cases.
4. Define the structured recipe schema.
5. Create the first small synthetic dataset for inventory and recipes.
6. Implement the standalone Python baseline recommendation engine.
7. Create tests for ranking logic and edge cases.
8. Only then begin the backend persistence layer and iOS UI flows.

Current stop point: Do not start by training a neural network. First make the data model and transparent recommendation baseline work on a small, understandable dataset.

Appendix A. Glossary

Term	Meaning
Inventory lot	A specific purchased quantity of an ingredient with its own dates and state.
Recommendation engine	The component that scores and ranks recipes or actions for a user.
Decision support	The layer that explains recommendations and exposes trade-offs.
Baseline	A simple, transparent system used as the reference point for later ML models.
Feature	A structured value used by a scoring or ML model, such as expiry urgency or cooking-time fit.
Cold start	The situation where a new user has little or no historical behavior data.
Learning loop	The process of collecting interactions, updating signals, training models and evaluating changes.
RAG / retrieval	Retrieving relevant structured or semantic context before generating a language-model response.
Online update	A small, frequent update to user state or model signals based on recent behavior.
Offline training	Training a model separately from the live application using collected historical data.

Appendix B. Example End-to-End Scenario

The following scenario illustrates how the planned components eventually work together. It is a target behavior, not an MVP requirement.

Inventory

- Spinach: 200 g, expires tomorrow
- Paneer: 250 g, expires in 2 days
- Tomatoes: 4, expires in 4 days
- Rice: 1 kg, long shelf life

User profile

- Frequently chooses Indian food
- Usually prefers meals under 30 minutes
- Has cooked paneer dishes positively before

System

1. Finds recipes compatible with current inventory.
2. Calculates ingredient match and expiry urgency.
3. Adds user preference and cooking-time signals.
4. Estimates expected leftovers.
5. Ranks candidates.
6. Produces reasons for the top options.
7. User cooks the top recipe and rates it positively.
8. The cooking event becomes a new learning signal.

Target outcome: The next recommendation should use the new behavior signal, but should still balance it with expiry urgency, ingredient availability, current context and exploration.

EVERYDAY KITCHEN

Build the product first. Add intelligence where it creates real value.

Working blueprint - Version 1.0